

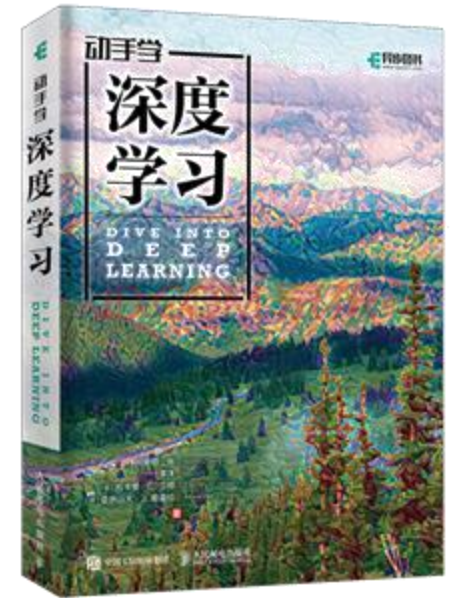
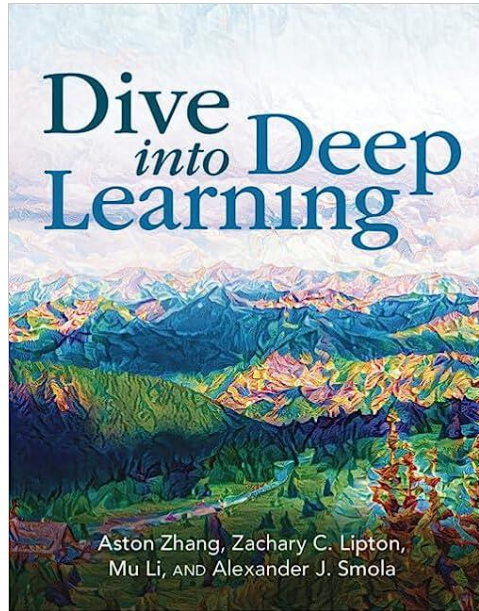
Multi-Layer Perceptron

Qian Wang

School of Biomedical Engineering
ShanghaiTech University

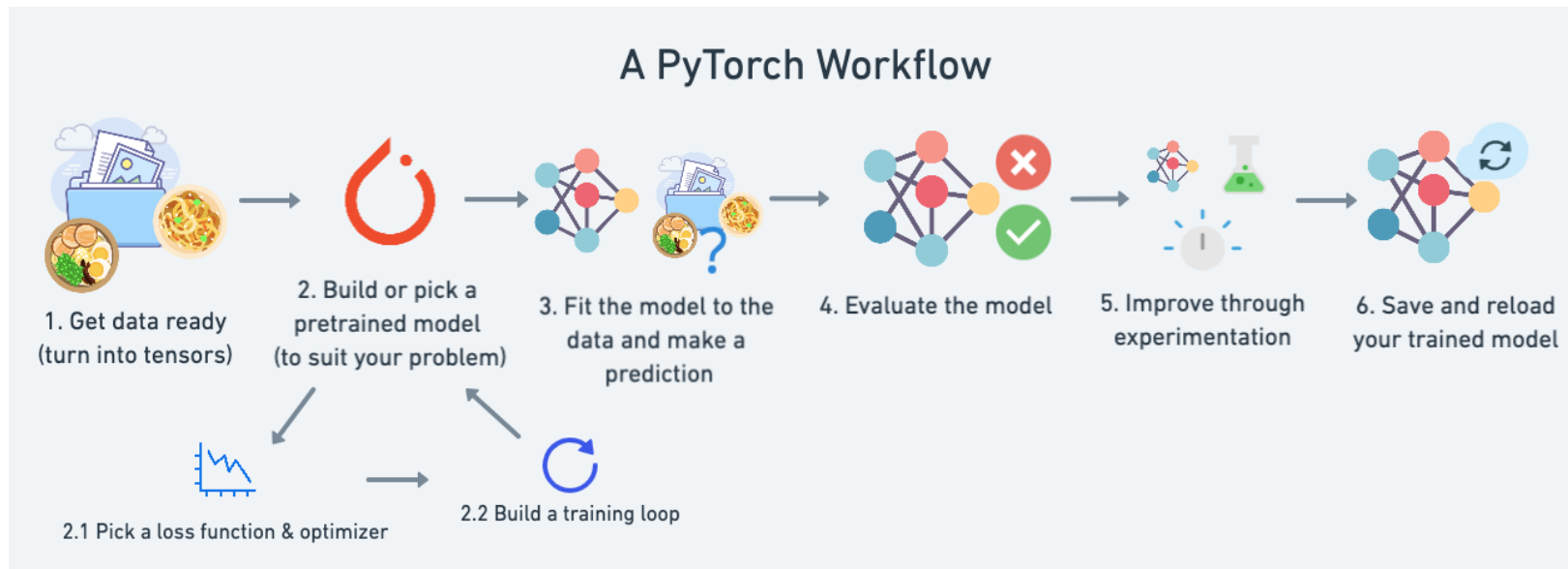
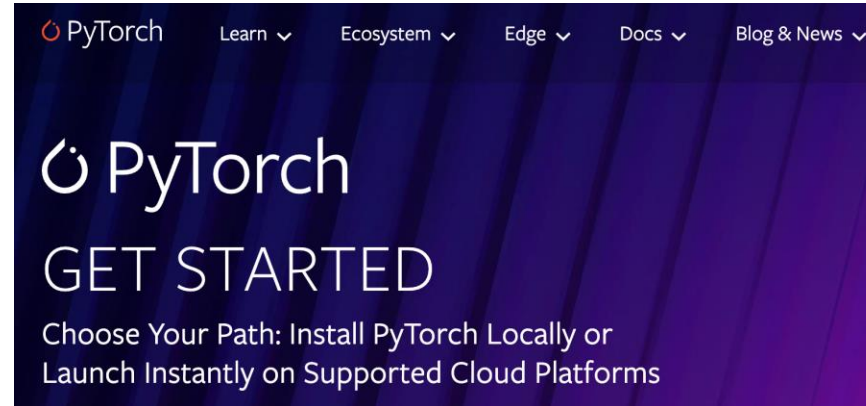
Materials

- For deep learning part, we will largely follow the book of Dive into Deep Learning
 - <http://d2l.ai/index.html>



Materials

- <https://pytorch.org/>



Linear Regression Model

- Assume a variable y can be expressed as a weighted sum of a set of variables $\mathbf{x}$, given some noise can be observed.
 - Example: to estimate the prices of houses based on areas and ages:

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b.$$

- To develop such a model, we usually need a training dataset at hand, which consists of real sales data of price, area, and age.
 - Each row is an example (or data point, instance, sample, etc.);
 - Predict from features to targets.

	Price	Area	Age
Sample – 1	...	...	...
Sample – 2	...	...	...
Sample – 3	...	...	...
...	...	...	...

Linear Regression Model

- Model parameters

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b.$$

The diagram shows the equation $\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b.$ with two red arrows pointing from the word "Weights" to the coefficients w_{area} and w_{age} , and one red arrow pointing from the word "Bias" to the constant term b .

- Weight: to determine the influence of each feature on prediction;
 - Bias: the value for the case when all features are zero.
- Training: determine weights and bias in accordance to sample features/labels.
- Testing: infer labels from sample features, with trained weights.

Vector Notation

- When inputs consist of d features, we express our prediction $\hat{y}$ by:

$$\hat{y} = w_1x_1 + \dots + w_dx_d + b.$$

- In vector form:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

- Design the feature matrix for all samples, where each row is a sample and each column for a feature, then we have

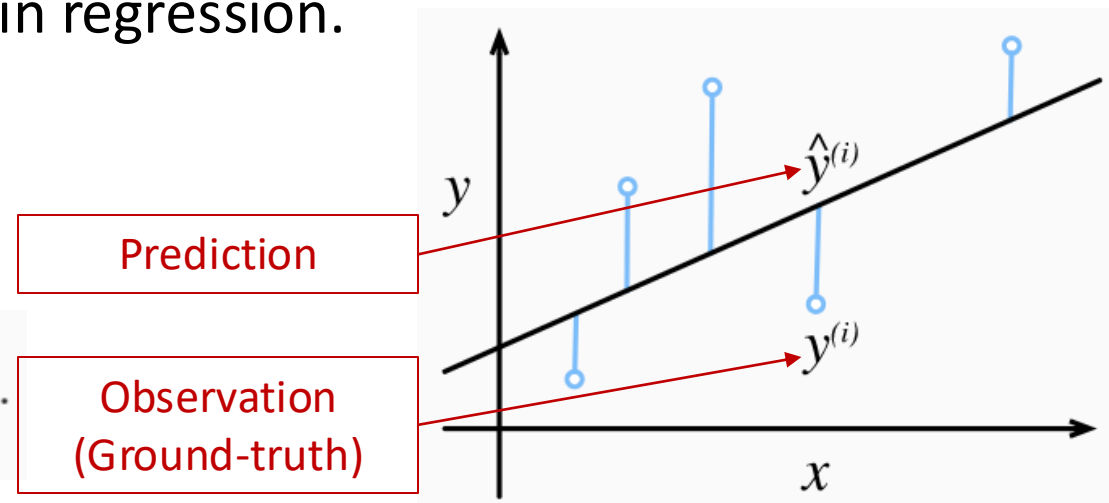
$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b,$$

Loss Function

- Before determining model parameters (i.e., weights and bias), a basic question is how good the model is.
 - We therefore need a quality measure for some given model.
 - **Loss function**: to quantify the distance between the real and predicted value of target.
 - Squared error: a most popular loss function in regression.

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2.$$

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2.$$



Solution to Linear Regression Model

- With a loss function for model quality measuring, we can solve for model parameters, i.e., to minimize the total loss across all training samples:

$$\mathbf{w}^*, b^* = \operatorname{argmin}_{\mathbf{w}, b} L(\mathbf{w}, b).$$

- Example: with squared error as loss function, we have **analytic solution**:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

- In practice, we usually adopt sophisticated **numeric optimization** method (e.g., gradient descent).

Some Math Background: Derivative

- Suppose a function with scalar input and output. The derivative is defined below if this limit exists:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- If $f'(a)$ exists, f is said to be differentiable at a .
 - $f'(x)$ is the instantaneous rate of change of $f(x)$ with respect to x .
- Be aware of many equivalent notations for derivative:

$$f'(x) = y' = \frac{dy}{dx} = \frac{df}{dx} = \frac{d}{dx}f(x) = Df(x) = D_x f(x)$$

Gradient

- Let $y = f(x_1, x_2, \dots, x_n)$, or $y = f(\mathbf{x})$ be a function with n variables. The **partial derivative** of y with respect to its i -th parameter x_i is:

$$\frac{\partial y}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}, \quad \text{or} \quad \frac{\partial y}{\partial x_i} = \frac{\partial f}{\partial x_i} = f_{x_i} = f_i = D_i f = D_{x_i} f.$$

- Concatenate partial derivatives of a multivariate function with respect to all its variables, and obtain the **gradient** vector of the function:

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right]^\top$$

- Some important gradients in vector form:

- For all $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\nabla_{\mathbf{x}} \mathbf{A} \mathbf{x} = \mathbf{A}^\top$,
- For all $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{A} = \mathbf{A}$,
- For all $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{A} \mathbf{x} = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$,
- $\nabla_{\mathbf{x}} \|\mathbf{x}\|^2 = \nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{x} = 2\mathbf{x}$.

Chain Rule

- Suppose that $y = f(g(x))$ and that the underlying functions $y = f(u)$ and $u = g(x)$ are both differentiable, the chain rule states that:

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}.$$

- In multi-variate case of $y = f(\mathbf{u})$, $\mathbf{u} = g(\mathbf{x})$ (e.g., $\mathbf{u}$ and $\mathbf{x}$ are vectors), then the chain rule states that:

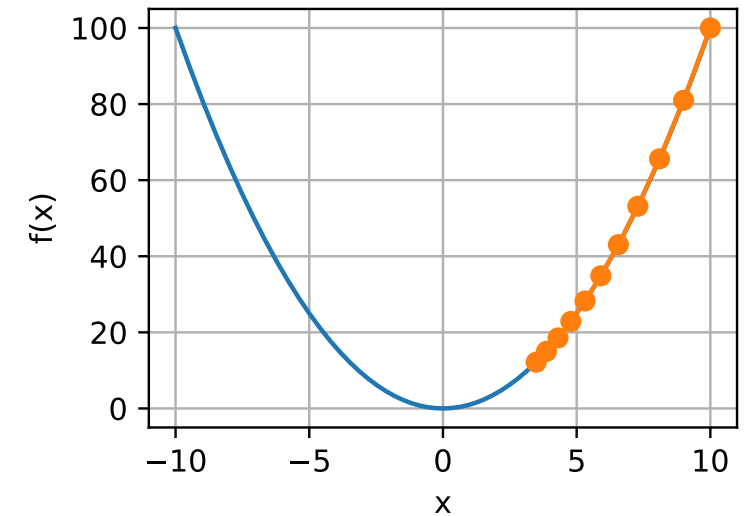
$$\frac{\partial y}{\partial x_i} = \frac{\partial y}{\partial u_1} \frac{\partial u_1}{\partial x_i} + \frac{\partial y}{\partial u_2} \frac{\partial u_2}{\partial x_i} + \dots + \frac{\partial y}{\partial u_m} \frac{\partial u_m}{\partial x_i}$$

$$\nabla_{\mathbf{x}} y = \mathbf{A} \nabla_{\mathbf{u}} y$$

A: Jacobian Matrix (in fact, the transposed version)

Gradient Descent

- Gradient Descent: minimizes the loss function with respect to the model's parameters, by iteratively moving in the direction of steepest descent as defined by the negative gradient.
 1. Start with an initial value for x ;
 2. Evaluate the loss function $y = f(x)$;
 3. Compute the derivative $\frac{df}{dx}$ at the current value of x ;
 4. Update the value of x using the gradient descent update rule: $x_{\text{new}} = x - \alpha \frac{df}{dx}$;
 5. Repeat steps 3 to 4 until convergence is reached.



Stochastic Gradient Descent

- Back to the linear regression problem, computing the gradient of the loss function over the entire dataset can be computationally expensive and time-consuming.
- **Stochastic Gradient Descent (SGD)**: approximates the true gradient using a small batch of data points randomly sampled from the dataset at each iteration.

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b)$$



$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b).$$

Stochastic Gradient Descent

- SGD:

- Instead of passing over all training samples, we can make an update with a randomly selected **minibatch** of samples.
- We can also control optimization speed by adjusting **learning rate**.

The diagram illustrates the Stochastic Gradient Descent (SGD) update equation. The main equation is:

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b).$$

Annotations in the diagram:

- A red box labeled "Learning Rate" points to the η in the numerator of the fraction.
- A red box labeled "Batch Size" points to the $|\mathcal{B}|$ in the denominator of the fraction.
- A black arrow points from the fraction down to the expanded vector and bias update equations.

The expanded update equations are:

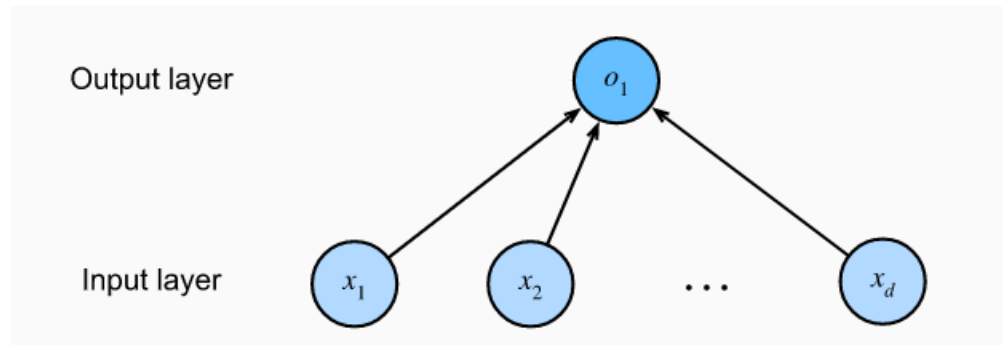
$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{\mathbf{w}} l^{(i)}(\mathbf{w}, b) = \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right),$$
$$b \leftarrow b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_b l^{(i)}(\mathbf{w}, b) = b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right).$$

Summary of Training Steps

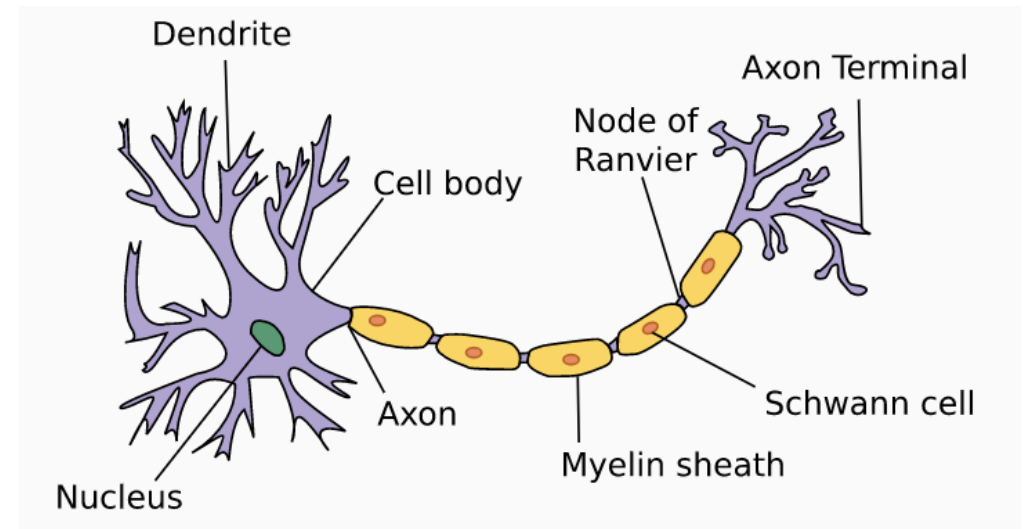
1. Prepare training samples, including their features and targets;
2. Design a loss function, to measure the fitness of the model;
3. Specify hyperparameters (batch size, learning rate, etc.);
4. Initialize model parameters (e.g., with random values);
5. Iteratively sample minibatches, and update model parameters in the direction of negative gradient;
6. Finish after training for some predetermined number of iterations (or until some other stopping criteria are met).
 - Now we should have estimated model parameters $\hat{\mathbf{w}}, \hat{b}$.

From Linear Regression to Neural Network

- Linear regression model can be perceived in a graph structure, which depicts a single-layer neural network architecture.
 - Number of inputs, or feature dimensionality, is d ;
 - Number of outputs is 1;
 - Number of layers is 1.



Single-Layer Neural Network



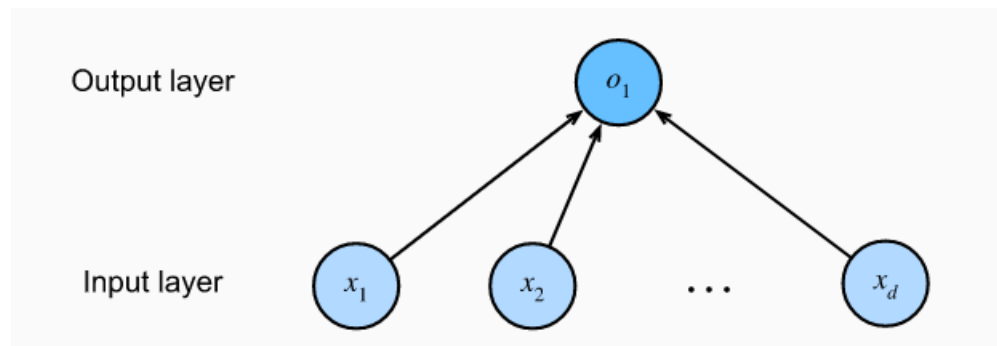
Similar to real neuron in real neural system!

Artificial Neural Network

- Artificial Neural Network (ANN) is a data-driven model inspired by the structure and function of the human brain.
 - Node: The basic unit of an ANN, analogous to a biological neuron. Each neuron receives one or more inputs, processes them, and produces an output.
 - Layer: input layer, hidden layer, output layer.
 - Weights and biases: Each connection from one neuron to another has an associated weight. Bias allows the node to shift the output.
 - Training Process: forward pass, loss function, backward pass.

Forward and Backward Propagation

- Forward propagation: It computes the output from the inputs and stores any necessary context that might be required for the backward pass.
- Backward propagation: Each operation has an associated backward function that knows how to compute the gradient of the output with respect to its inputs. This backward function is the derivative of the forward operation.



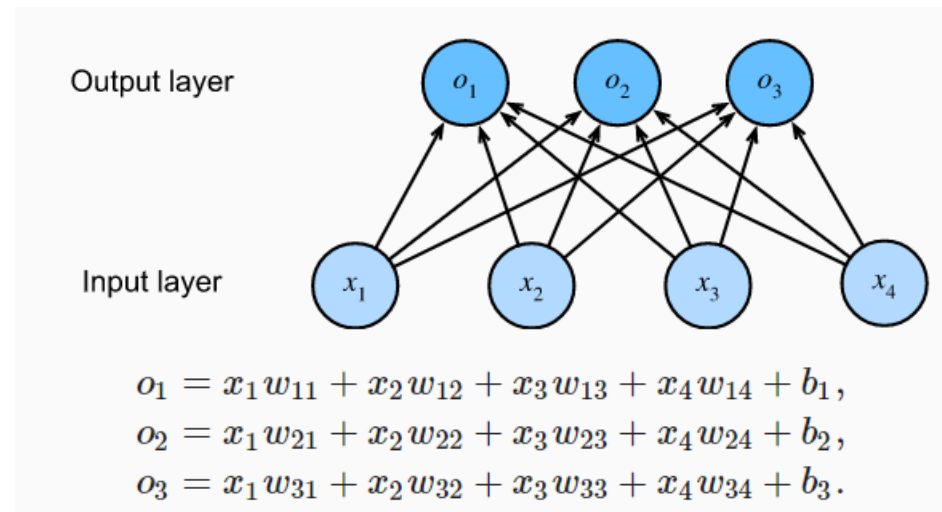
One-hot Encoding

- From regression to classification: in practice, we are often interested in classification.
- **One-hot encoding**: use a vector with as many components as we have categories for classification.
 - The component corresponding to particular sample's category is set to 1 and all other components are set to 0.
 - Example: Let us assume that each image belongs to one among the three categories “cat”, “chicken”, and “dog”.

1	0	0
0	1	0
0	0	1
cat	chicken	dog

Softmax Regression

- To solve the classification problem, we now need a model with multiple outputs, each of them corresponding with a class.
 - It can still be a single-layer neural network, which is simple;
 - The output layer can be described as fully-connected layer.



In vector form, we have
 $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$

Logits

- The outputs of the linear transformation of input features (i.e., the raw, unnormalized predictions of the model especially before applying the softmax function) are referred to as **logits**.
 - For a model predicting multiple classes, these logits represent the raw prediction scores for each class. So we can interpret them as probabilities.
 - However, directly interpreting logit as a probability faces challenges:
 - On one hand, nothing constrains these numbers to sum to 1.
 - On the other hand, depending on the inputs, they can take negative values.
 - We must guarantee that (even on new test data), they will be nonnegative and sum up to one.

Softmax Operation

- The softmax function precisely transforms logits $\mathbf{o}$ such that they become nonnegative and sum to one.
 - To keep the model differentiable, we first exponentiate each logit (ensuring non-negativity) and then divide by their sum (ensuring that they sum to one):

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}.$$

- Note that the softmax operation does not change the ordering among the logits $\mathbf{o}$, which are simply the pre-softmax values that determine the probabilities assigned to each class.

$$\operatorname{argmax}_j \hat{y}_j = \operatorname{argmax}_j o_j.$$

Cross-Entropy Loss

- Note $\hat{\mathbf{y}}$ can be interpreted as conditional probabilities of each class given any input $\mathbf{x}$.
 - Suppose the entire dataset $\{\mathbf{X}, \mathbf{Y}\}$ has n examples, where the example indexed by i consists of a feature vector $\mathbf{x}^{(i)}$ and a one-hot label vector $\mathbf{y}^{(i)}$.
 - We can compare the estimates with reality by checking how probable the actual classes are according to our model, given the features:

$$P(\mathbf{Y} | \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}).$$

Cross-Entropy Loss

- To maximize the previous objective function, it is equivalent to minimize the negative log-likelihood:

$$-\log P(\mathbf{Y} | \mathbf{X}) = \sum_{i=1}^n -\log P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}) = \sum_{i=1}^n l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}),$$

- Assuming q classes, the loss function l is called the **cross-entropy loss**.

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j.$$

How does it work?

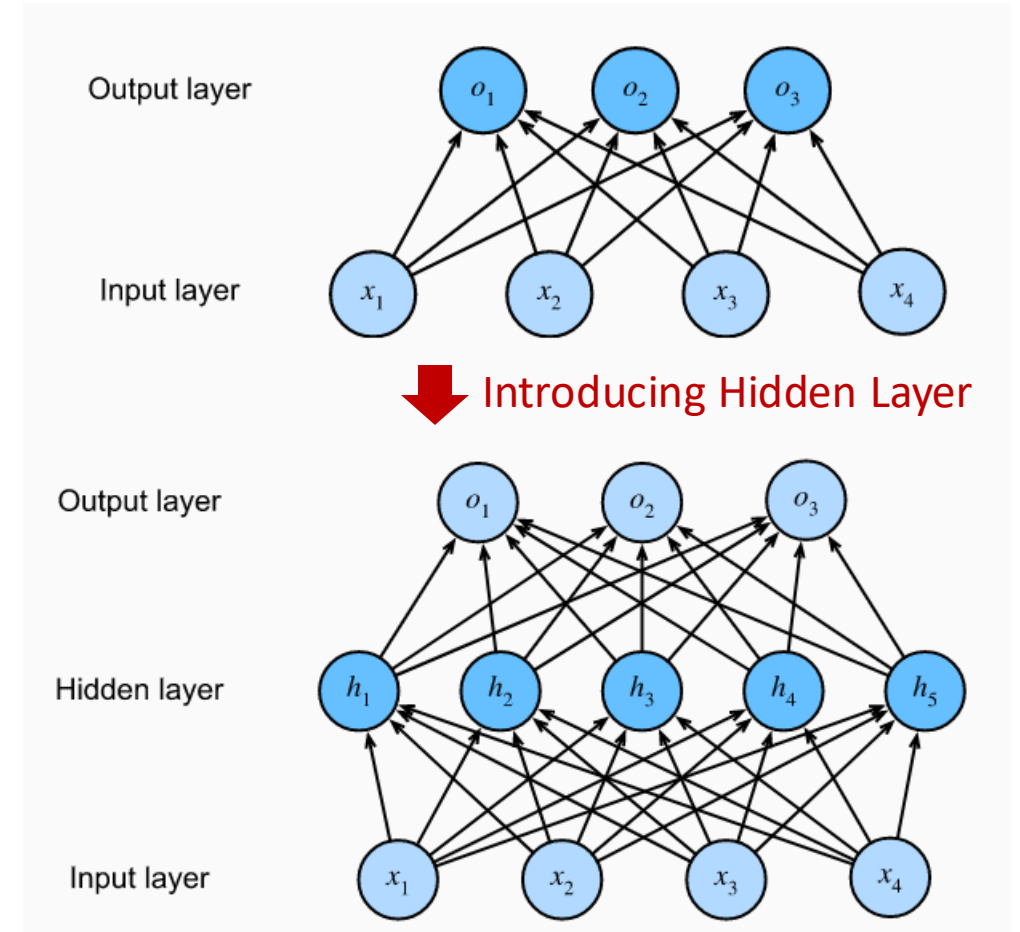
- True $\mathbf{y}$: (1, 0, 0)
- Predicted $\hat{\mathbf{y}}$: (0.5, 0.5, 0)

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= -\sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \\ &= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \\ &= \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j. \end{aligned}$$

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j = \text{softmax}(\mathbf{o})_j - y_j.$$

Multi-layer Perceptron (MLP)

- Linearity is weak to represent real-world relationship.
- To increase the complexity of the neural network, we can stack many fully-connected layers on top of each other.
 - Each layer feeds into the layer above it, until we generate outputs.
 - We can think of the first $L - 1$ layers as our representation and the final layer as our linear predictor.



This MLP has 4 inputs, 3 outputs, 2 layers. Its hidden layer contains 5 hidden nodes.

From Linear to Nonlinear

- Simply stacking hidden layers doesn't contribute.
 - It is still a linear representation between input $\mathbf{X}$ and output $\mathbf{O}$, given the hidden layer $\mathbf{H}$:

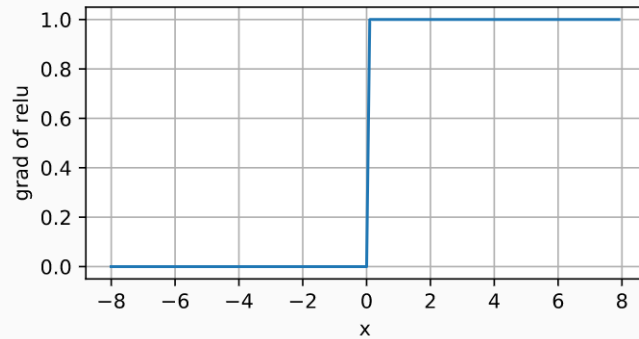
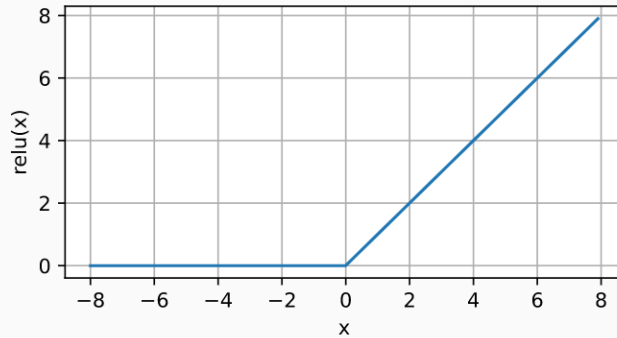
$$\begin{aligned} \mathbf{H} &= \mathbf{XW}^{(1)} + \mathbf{b}^{(1)}, \\ \mathbf{O} &= \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}. \end{aligned} \quad \longrightarrow \quad \mathbf{O} = (\mathbf{XW}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW} + \mathbf{b}.$$

- A key missing component here is the **nonlinear activation function σ** .

$$\begin{aligned} \mathbf{H} &= \sigma(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)}), \\ \mathbf{O} &= \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}. \end{aligned}$$

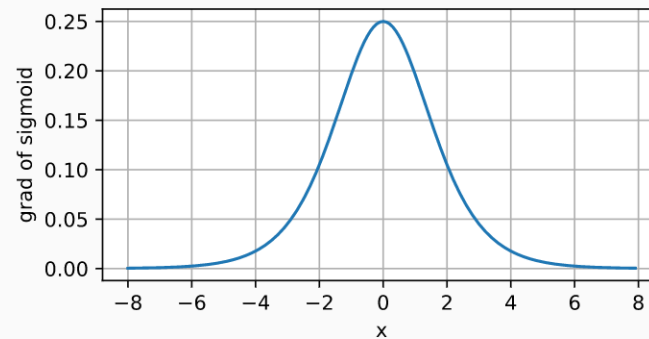
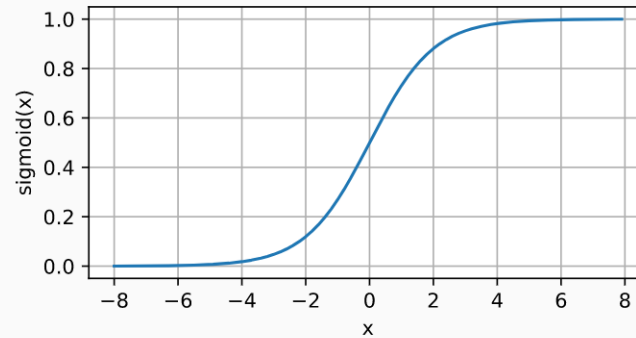
- For each hidden node, an activation function σ is applied.
- Can easily generate to more hidden layers.

Activation Function



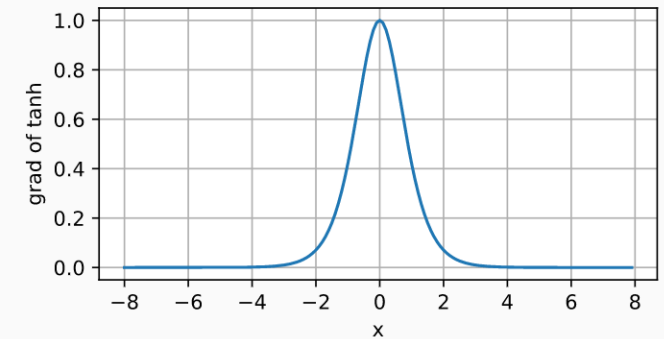
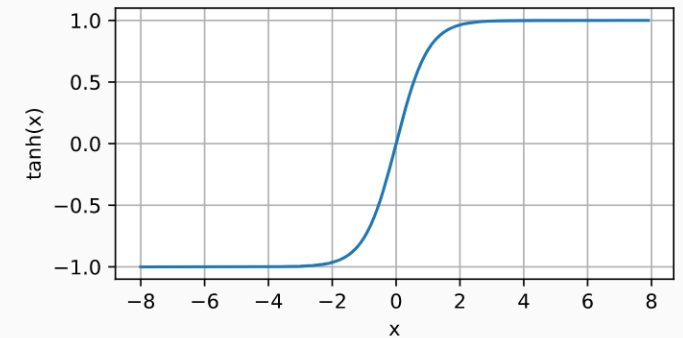
$$\text{ReLU}(x) = \max(x, 0).$$

ReLU (rectified linear unit)



$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

Sigmoid

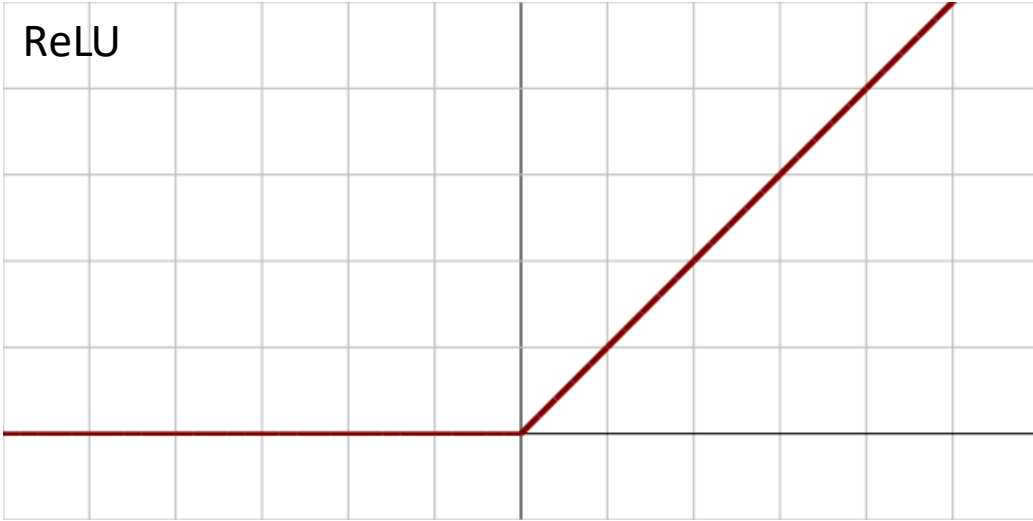


$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$

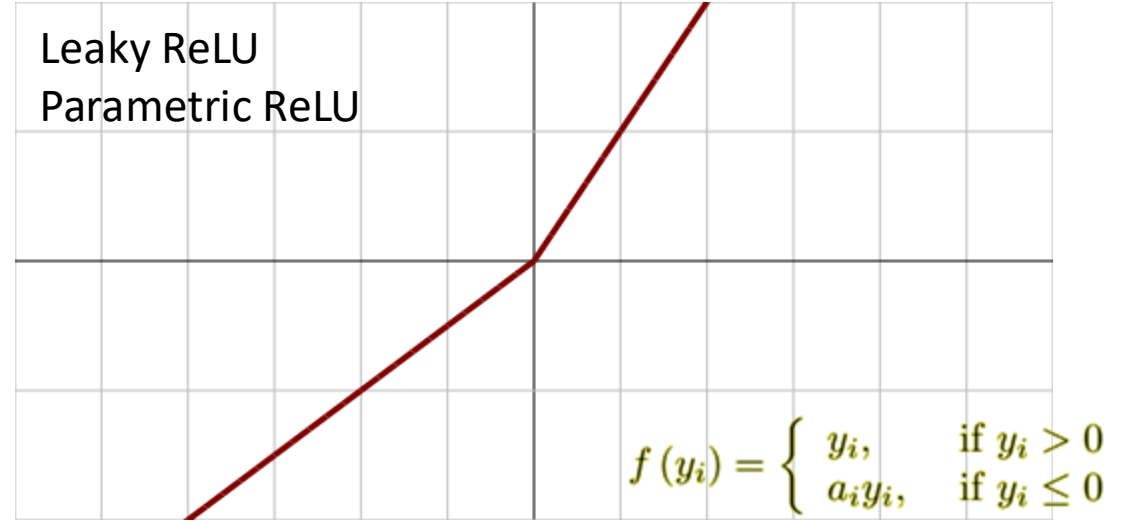
tanh (hyberbolic tangent)

Variants of ReLU

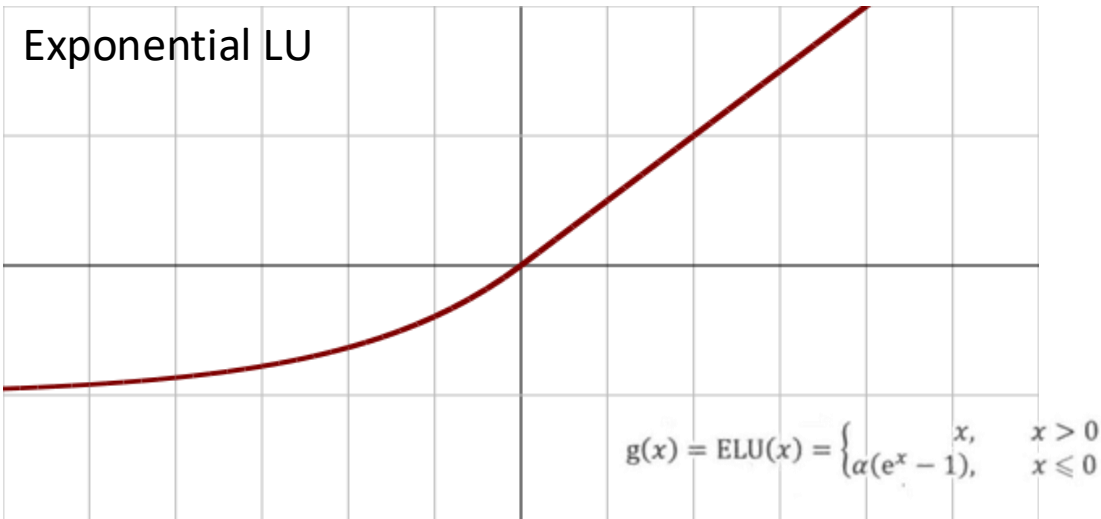
ReLU



Leaky ReLU
Parametric ReLU



Exponential LU



Universal Approximator

- It is widely known that MLPs are universal approximators.
 - Even with a single-hidden-layer network, given enough nodes (possibly absurdly many), and the right set of weights, we can model any function, though actually learning that function is hard.
 - Moreover, just because a single-hidden-layer network can learn any function does not mean that you should try to solve all of your problems with single-hidden-layer networks.
 - In fact, we can approximate many functions much more compactly by **using deeper (vs. wider) networks**.

Generalizability: Impact Factors

- The number of tunable parameters.
 - When the number of tunable parameters, sometimes called the degrees of freedom, is large, models tend to be more susceptible to overfitting.
- The values taken by the parameters.
 - When weights can take a wider range of values, models can be more susceptible to overfitting.
- The number of training examples.
 - It is trivially easy to overfit a dataset containing only one or two examples even if your model is simple. But overfitting a dataset with millions of examples requires an extremely flexible model.

Regularization

- The very important task is to design a proper loss function for the network.
 - The loss in general tells you how similar two samples are.
 - Regularization is essential – we can always mitigate overfitting by going out and collecting more training data. But it can be costly, time consuming, or entirely out of our control, making it impossible in the short run.

Weight Decay

- Weight decay (commonly called L_2 regularization), might be the most widely-used technique for regularizing parametric machine learning models.
 - It is motivated by the basic intuition that among all functions f , the function $f = 0$ (assigning the value 0 to all inputs) is in some sense the simplest.
 - So we can measure the complexity of a function by its distance from zero.
 - One simple interpretation might be to measure the complexity of a linear function $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ by some norm of its weight vector, e.g., $\|\mathbf{x}\|^2$, which is added as a penalty term to the loss.

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\mathbf{w}^T \mathbf{x}^{(i)} + b - y^{(i)} \right)^2 \quad \longrightarrow \quad L(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2,$$

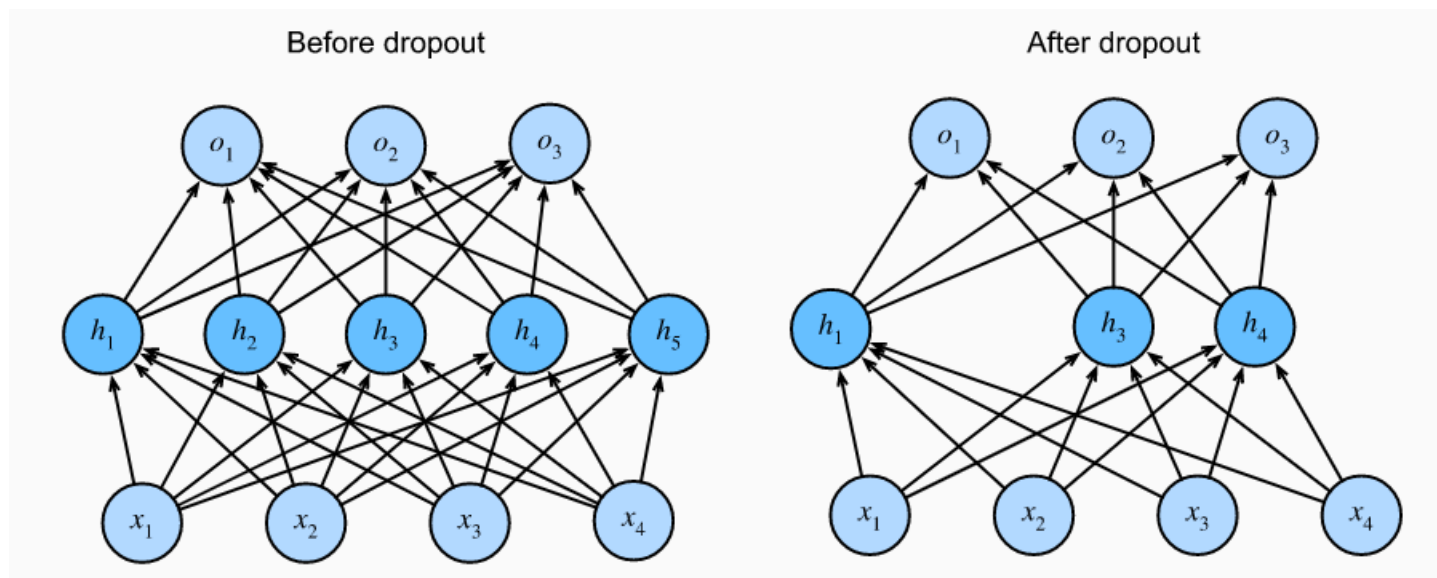
Minimizing the prediction loss

Minimizing the sum of the prediction
loss and the penalty term

Dropout

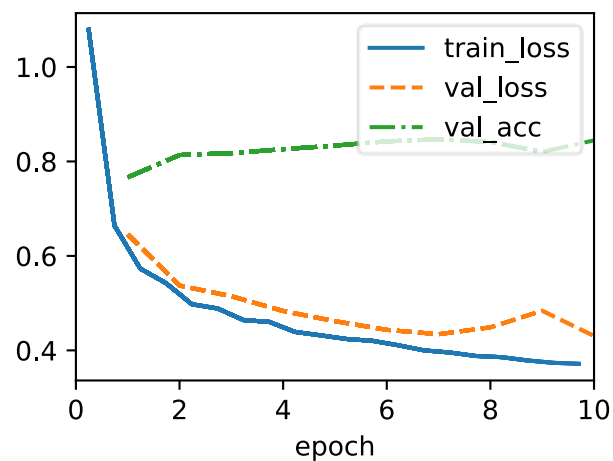
- In training, we dropout node activation h with the probability p .
- In testing, we disable dropout.

$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$



MLP: Implementation

```
class MLP(d2l.Classifier):  
    def __init__(self, num_outputs, num_hiddens, lr):  
        super().__init__()   
        self.save_hyperparameters()  
        self.net = nn.Sequential(nn.Flatten(), nn.LazyLinear(num_hiddens),  
                                  nn.ReLU(), nn.LazyLinear(num_outputs))  
  
model = MLP(num_outputs=10, num_hiddens=256, lr=0.1)  
trainer.fit(model, data)
```



Computational Graph

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x},$$

$$\mathbf{h} = \phi(\mathbf{z}).$$

$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h}.$$

$$L = l(\mathbf{o}, y).$$

$$s = \frac{\lambda}{2} \left(\|\mathbf{W}^{(1)}\|_{\text{F}}^2 + \|\mathbf{W}^{(2)}\|_{\text{F}}^2 \right),$$

$$J = L + s.$$

Assume hidden layer does not include a bias term;

Apply activation function ϕ ;

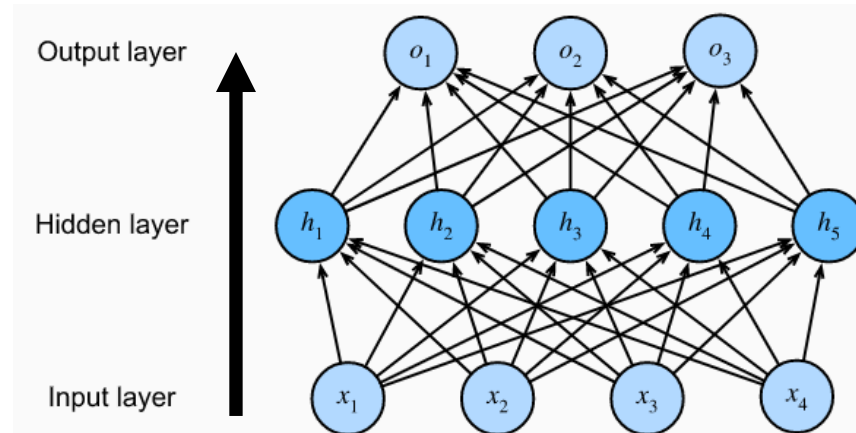
Obtain the logits from the output layer;

Calculate the loss term, i.e., for a single sample;

Assume the regularization term;

Reach the objective function in the final.

Forward
Propagation



Computational Graph

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x},$$

$$\mathbf{h} = \phi(\mathbf{z}).$$

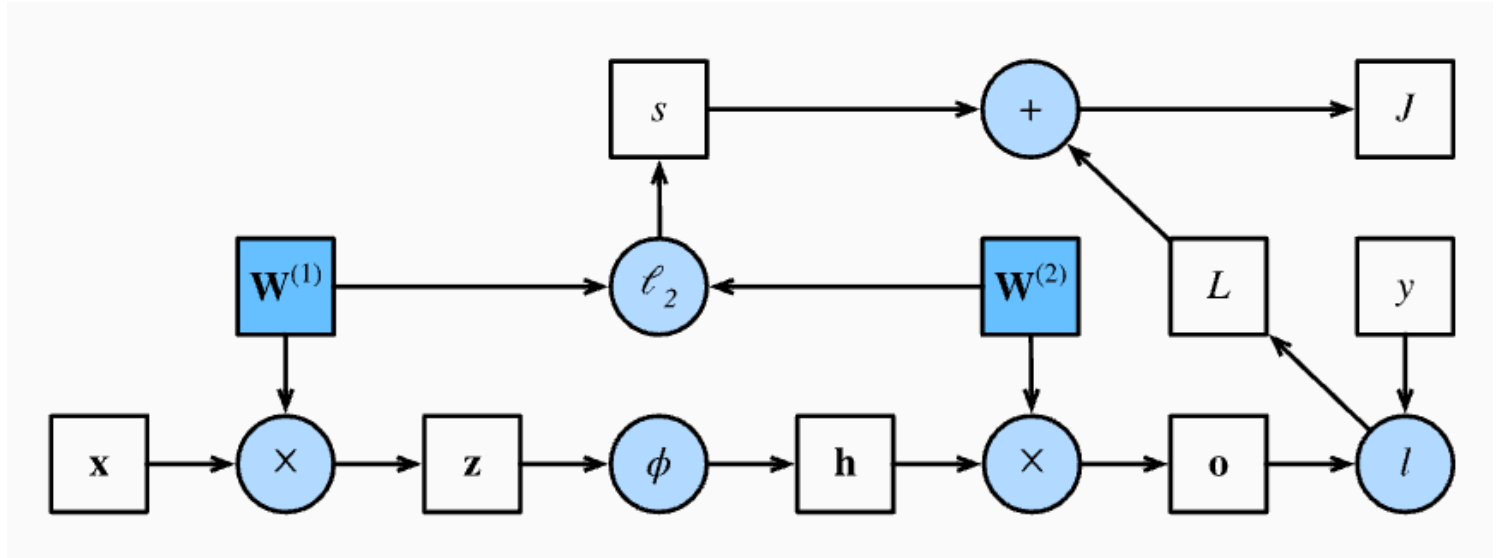
$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h}.$$

$$L = l(\mathbf{o}, y).$$

$$s = \frac{\lambda}{2} \left(\|\mathbf{W}^{(1)}\|_{\text{F}}^2 + \|\mathbf{W}^{(2)}\|_{\text{F}}^2 \right),$$

$$J = L + s.$$

Forward
Propagation



Computational graph of forward propagation

- PyTorch builds a dynamic **computation graph** represents the flow of data and operators during the forward pass.
- In PyTorch, a **tensor** is a multi-dimensional array that is a primary data structure used to store data.
- Each **operator** creates a node in the graph, and each tensor becomes a leaf or intermediate node.

Backproagation

- **Backward pass**: When you call `.backward()` on a tensor, PyTorch performs the backward pass, or **backpropagation**, through this computation graph. This system is called Automatic Differentiation (**autograd**).
 - Each tensor operation (e.g., addition, multiplication) knows how to compute its gradient with respect to its inputs.
 - For each operation in the graph, PyTorch applies the chain rule to calculate the gradient of the output with respect to the inputs.
 - Gradients are accumulated in the `grad` attribute of each tensor. This allows for the gradients to be updated iteratively, which is essential for training models using optimization algorithms like stochastic gradient descent (SGD).

Summary

- MLP is a class of artificial neural networks used for various tasks such as classification and regression.
 - MLPs consist of input layer, one or more hidden layers, and an output layer. Each layer is fully connected to the next one.
 - Each connection between nodes has an associated weight, and each node (except those in the input layer) has a bias.
 - Activation functions introduce non-linearity into the model.
 - MLPs are typically trained using backpropagation and a variant of gradient descent to minimize the error between the predicted and actual outputs.

Thank You!